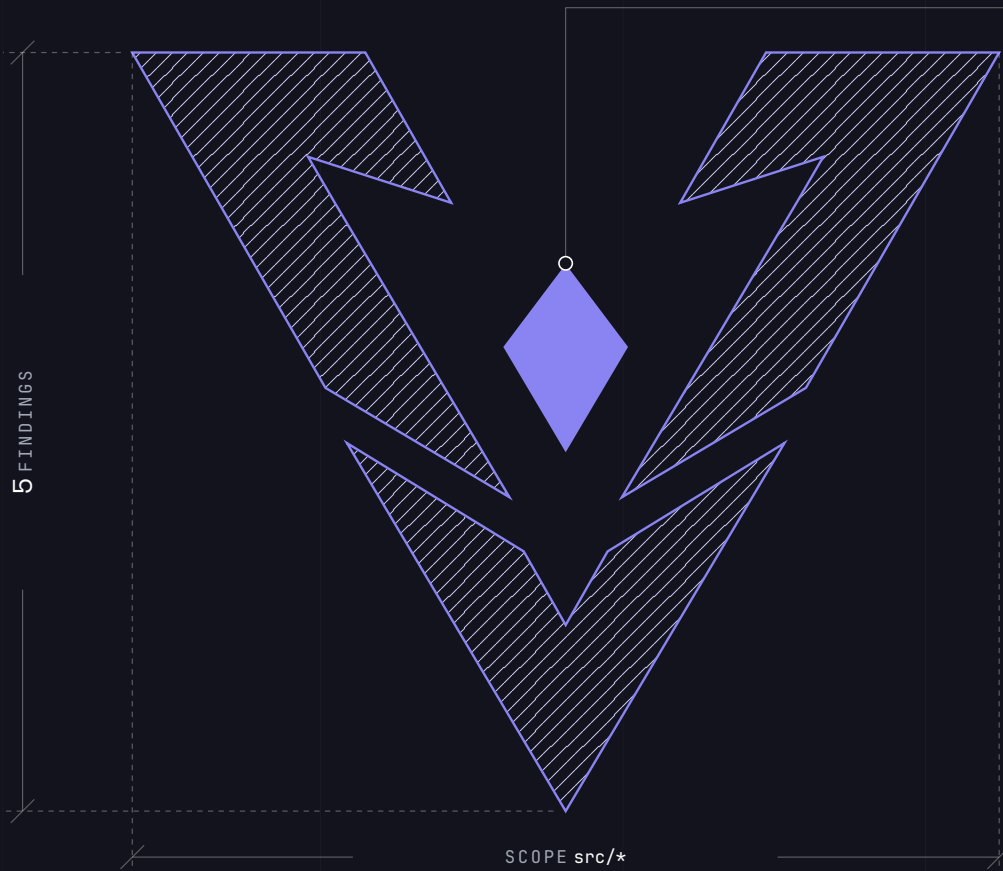


COMMIT 5E2EC27



SECURITY REVIEW

UniVoucher.

March 11th 2026 - March 13th 2026

Contents.

01	About Valves Security	3
02	Disclaimer	3
03	Risk Classification	3
04	Protocol Summary	4
05	Executive Summary	4
06	Findings	5
	■ High Findings	5
	[H-01] Partner address excluded from signed message enables front-running theft	5
	■ Medium Findings	6
	[M-01] Fee-on-transfer tokens cause undercollateralized slots	6
	[M-02] Fixed-liability accounting is incompatible with rebasing and balance-mutating tokens	7
	■ Low/Info Findings	7
	[L-01] Dust deposit front-running blocks any <code>slotId</code>	7
	[I-01] Cross-Deployment <code>cardId</code> Collision	8

SECTION 01

About Valves Security.

Valves Security is an independent smart contract security firm founded by Veselin and Valeri. We work with security researchers specializing in Solidity, Rust and DAML. Every engagement is led by both co-founders with senior security researchers: we review the code line by line, substantiate every reported issue and remain involved through remediation.

To request a security review, visit valvessecurity.com or contact @ValvesSec on X.

SECTION 02

Disclaimer.

This report reflects a time-boxed review of the code at the commit listed in the Executive Summary, limited to the security of the files in scope. A security review cannot guarantee the absence of vulnerabilities, and this report is neither an endorsement of the project, its team or any associated token, nor investment advice. Changes made after the reviewed commit may introduce new issues; we recommend follow-up reviews and an ongoing bug bounty program.

SECTION 03

Risk Classification.

LIKELIHOOD / IMPACT	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
High	■ Critical	■ High	■ Medium
Medium	■ High	■ Medium	■ Low
Low	■ Medium	■ Low	■ Low

Impact

- **High** — leads to a significant material loss of assets in the protocol or a group of users.
- **Medium** — leads to a moderate material loss of assets in the protocol, moderately harms a group of users or causes some disruption of the protocol's functionality.
- **Low** — no funds at risk.

Likelihood

- **High** — the attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- **Medium** — only a conditionally incentivized attack vector, but still relatively likely.
- **Low** — has unlikely assumptions or requires a significant stake by the attacker.

SECTION 04

Protocol Summary.

UniVoucher introduces a decentralized crypto gift card primitive that allows any token to be distributed via simple voucher codes secured by on-chain smart contracts. By removing custodial control and enabling physical or digital distribution of crypto assets, the protocol bridges Web3 infrastructure with real-world gifting and reward systems.

SECTION 05

Executive Summary.

OVERVIEW

PROJECT UniVoucher	REVIEW PERIOD March 11th 2026 - March 13th 2026
COMMIT 5e2ec2752829c74680a00f9af847708c74b25a2b	

SCOPE

ID	FILES IN SCOPE
01	src/*

SECTION 06

Findings.

■ High Findings

[H-01] Partner address excluded from signed message enables front-running theft

HIGH

Description

The redemption signature covers only `cardId` and `to`, but not `partner`. Any caller can take a valid signature and submit it with an arbitrary `partner` address.

```
1  /**
2   * @dev Redeem a gift card using its ID and signature.
3   * @param cardId The ID of the gift card to redeem
4   * @param to The address where to send the redeemed tokens
5   * @param signature A signature proving ownership of the card secret
6   * @param partner Optional partner address to receive 1% of the card amount (use address(0) for
   *    ↪ no partner)
7   */
8  function redeemCard(string memory cardId, address payable to, bytes memory signature, address
   *    ↪ payable partner) external nonReentrant {
9      // Code...
10
11     // Calculate partner fee (1% if partner is provided, minimum 1 wei)
12     uint256 partnerAmount = (partner != address(0)) ?
13         (slotCopy.tokenAmount / 100 > 0 ? slotCopy.tokenAmount / 100 : 1) : 0;
14
15     // Transfer the assets
16     if (slotCopy.tokenAddress == address(0)) {
17         // Transfer ETH
18         if (partnerAmount > 0) {
19             (bool sentPartner, ) = partner.call{value: partnerAmount}("");
20             require(sentPartner, "Failed to send ETH to partner");
21         }
22         (bool sent, ) = to.call{value: slotCopy.tokenAmount - partnerAmount}("");
23         require(sent, "Failed to send ETH");
24     } else {
25         // Transfer ERC20 tokens
26         if (partnerAmount > 0) {
27             IERC20(slotCopy.tokenAddress).safeTransfer(partner, partnerAmount);
28         }
29         IERC20(slotCopy.tokenAddress).safeTransfer(to, slotCopy.tokenAmount - partnerAmount);
30     }
31
32     emit CardRedeemed(cardId, slotId, to, slotCopy.tokenAddress, slotCopy.tokenAmount, partner,
   *    ↪ block.timestamp);
```

```
33 }
```

Impact

An MEV bot can front-run every redemption transaction, injecting itself as `partner` to steal 1% of the card value from every redemption on the protocol.

Proof of Concept

1. Alice broadcasts `redeemCard(cardId, to=Alice, sig, partner=address(0))`
2. Bob front-runs with `redeemCard(cardId, to=Alice, sig, partner=Bob)`
3. Bob receives `tokenAmount / 100`, Alice receives the remainder

Recommended Mitigation

Include `partner` in the signed message hash:

```
1 - bytes32 messageHash = keccak256(abi.encodePacked("Redeem card:", cardId, "to:", to));
2 + bytes32 messageHash = keccak256(abi.encodePacked("Redeem card:", cardId, "to:", to, "partner:",
  ↪ partner));
```

■ Medium Findings

[M-01] Fee-on-transfer tokens cause undercollateralized slots

MEDIUM

Description

`depositERC20` transfers `amount + fee` from the sender and records `amount` in the slot, but fee-on-transfer tokens deliver less than `amount + fee` to the contract.

Impact

The contract records more tokens than it holds. Redemptions and cancellations for the last slot will fail. This will lead to locking the last user's funds.

Proof of Concept

1. Alice deposits 1000 DEFTOKEN (0.2% transfer fee) — contract records `tokenAmount = 1000`
2. Contract actually receives 998 tokens (consider protocol fee == 0)
3. On redemption of a given slot, `safeTransfer(to, 1000)` reverts — funds are stuck

Recommended Mitigation

Measure the actual received balance and use that as `tokenAmount`:

```
1 + uint256 balanceBefore = IERC20(tokenAddress).balanceOf(address(this));
2   IERC20(tokenAddress).safeTransferFrom(msg.sender, address(this), totalRequired);
3 + uint256 actualAmount = IERC20(tokenAddress).balanceOf(address(this)) - balanceBefore - fee;
4 - _createCard(slotId, tokenAddress, amount, fee, message, encryptedPrivateKey);
5 + _createCard(slotId, tokenAddress, actualAmount, fee, message, encryptedPrivateKey);
```

[M-02] Fixed-liability accounting is incompatible with rebasing and balance-mutating tokens

MEDIUM

Description

Each voucher stores a fixed tokenAmount at creation time, and that value is later used for redemption and cancellation. The protocol never reconciles stored liabilities against the contract's live balance for the token.

Impact

If a token negatively rebases or otherwise reduces balances, stored voucher obligations can exceed the actual balance, causing redemption or cancellation failures.

Recommended Mitigation

Explicitly disallow rebasing and balance-mutating tokens, or redesign the accounting model so obligations are derived from actual balances rather than immutable nominal amounts stored at deposit time.

■ Low/Info Findings

[L-01] Dust deposit front-running blocks any slotId

LOW

Description

`_createCard` rejects a `slotId` that is already registered. Anyone can register any `slotId`. An attacker can watch the mempool and front-run a victim's large deposit with a 1-wei deposit to the same `slotId`.

Impact

The victim's transaction reverts. This can be repeated at near-zero cost to grief any user trying to create a card for somebody. Also, this harms the protocol as the fee from the big deposit (that was not executed) is not recorded.

Proof of Concept

1. Alice broadcasts `depositETH(slotId=X, amount=10 ETH, ...)`
2. Bob front-runs with `depositETH(slotId=X, amount=1 wei, ...)`
3. Alice's transaction reverts with "Slot ID already used"

Recommended Mitigation

Require a minimum voucher value to make the attacker's griefing unsustainable.

[I-01] Cross-Deployment cardId Collision

INFORMATIONAL

Description

`_generateCardId()` constructs a `cardId` by concatenating `versionPrefix`, `chainPrefix`, and `nextCardNumber`.

```
1  string memory cardId = string(abi.encodePacked(
2      versionPrefix,
3      chainPrefix,
4      _uintToString(nextCardNumber)
5  ));
```

Two contracts deployed with different but concatenation-ambiguous prefixes (“1”+“10” vs “10”+“1”) produce identical IDs for the same counter value. Additionally, two contracts with identical prefixes deployed on different chains will produce fully identical `cardId` sequences.

Impact

The same `cardId` can exist on two distinct contracts where we have got a different `versionPrefix` and `chainPrefix`.

Recommendation

Include `address(this)` in the `abi.encodePacked` call so each `cardId` is scoped to the specific contract instance, making it globally unique across all deployments:

```
1  function _generateCardId() internal returns (string memory) {
2      string memory cardId = string(abi.encodePacked(
3          versionPrefix,
4          chainPrefix,
5          _uintToString(uint256(uint160(address(this)))),
6          _uintToString(nextCardNumber)
7      ));
8      nextCardNumber++;
9      return cardId;
10 }
```